Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Course Project

Team leader: Hongyu Guo
Team members: Hongyu Guo (individual project)
GitHub: `[repository URL to be added]`

Abstract

We study whether a classifier can tell which large language model (LLM) wrote a piece of text. Using six model families from the public TXD-22 dataset (ChatGPT, Gemini, Claude, Llama, DeepSeek and Qwen; 17,988 responses to the same set of about three thousand questions), we train a convolutional network and a bidirectional LSTM from scratch. From the response alone the CNN reaches 87.5% test accuracy and the BiLSTM 84.6% (chance is 16.7%), and a TF-IDF logistic regression reaches 92.5%. The prompt carries no information in this dataset because every model answered the same questions. We also test transfer across question types and examine which signals the classifiers rely on.

1 Introduction

Text written by LLMs is now common, and different models may leave different traces in what they write. This project asks whether those traces are strong enough for a neural network to name the model that produced a response. We treat it as a six-way text classification problem and answer four research questions. RQ1 asks how well a CNN and an RNN identify the model from the response alone. RQ2 asks whether the user's prompt helps, by comparing input-only, output-only and input+output classifiers. RQ3 asks whether a classifier trained on one kind of question still works on another kind. RQ4 asks what the classifiers rely on, using stylometric features, distinctive n-grams and ablations.

2 Problem Definition and Dataset Curation

2.1 Task

Each example is a triple (LLM name, LLM input, LLM output). The input is a question, the output is the model's answer, and the label is the model family. The main task (RQ1) is to predict the label from the output text. With six balanced classes, a classifier that guesses has an expected accuracy of 16.7%. We report accuracy, macro-averaged F1, per-class F1 and a row-normalised confusion matrix for every experiment.

2.2 Data source and how it was collected

We use TXD-22, a public dataset released on Mendeley Data by Iqbal and Kashem under a CC BY 4.0 licence [1]. We did not generate any text ourselves. According to the dataset description, the authors wrote a hierarchical set of questions covering 30 academic and professional subjects and sent the same questions to 17 AI systems between January and March 2025, asking for answers of at most about 300 words. They also collected human-written answers and four kinds of mixed human/AI text, and they report normalising Unicode and whitespace and removing duplicates. The released CSV file has 66,000 rows in 22 classes of 3,000 rows each, with three columns: `Question`, `Answer` and `Source`. These map directly to the required (LLM_Input, LLM_output, LLM_name) triple. The description does not say which version of each model was used or whether answers came from an API or a chat interface, which limits what we can conclude (Section 6). The file contains only questions on academic subjects and model answers, so it has no private or sensitive information.

2.3 Selecting six models from the 22 classes

We kept the six classes named in our project plan: ChatGPT, Gemini, Claude, Llama, DeepSeek and Qwen. They come from six different developers (OpenAI, Google, Anthropic, Meta, DeepSeek and Alibaba), so the requirement of at least three LLM families is met with room to spare. Each of the six appears in the file under exactly one `Source` label, so no label mapping or merging was needed. We excluded the human-written and mixed-authorship classes because they are not LLMs. We excluded the other eleven AI classes to keep the

problem focused: several are products that can serve answers from more than one underlying model (for example Poe and Perplexity), and Gemma shares a developer with Gemini. The filter is applied first, so the other classes never influence the vocabulary, the split or the models.

2.4 Data audit

Table 1 summarises the subset. Before cleaning it has 18,000 rows, exactly 3,000 per model. There are no missing or empty responses. One Llama response is the spreadsheet error string `#NAME?`, and 11 responses are exact copies of another response from the same model. No response text is shared between two different models.

Table 1: The six-model subset. "Removed" counts rows dropped by cleaning. Words are runs of letters; tokens are counted with our tokenizer (Section 3). Mean, median and 99th percentile are per response.

Model	Raw	Removed	Kept	Train	Val	Test	Words	Tokens	Median	P99
ChatGPT	3000	0	3000	2100	450	450	245	291	286	479
Gemini	3000	8	2992	2093	450	449	266	322	355	579
Claude	3000	0	3000	2100	450	450	298	344	337	538
Llama	3000	1	2999	2099	449	451	210	244	240	351
DeepSeek	3000	3	2997	2098	450	449	267	336	336	470
Qwen	3000	0	3000	2100	450	450	219	266	271	395
Total	18000	12	17988	12590	2699	2699				

Length. Figure 1 shows the distribution of response length. The median response has 301 tokens and 99% of responses have at most 504 tokens. Length differs by model: Llama is the shortest on average (244 tokens) and Claude the longest (344). Length is therefore a possible cue, which we test in Section 5.3.

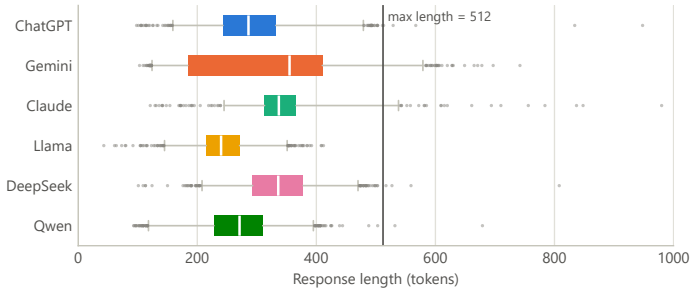


Figure 1: Response length in tokens for each model. Boxes show the interquartile range with the median, whiskers the 1st to 99th percentile, and dots the remaining responses. The vertical line is the maximum sequence length used by the neural models.

Prompt overlap. This is the most important property of the dataset. The subset contains 3,082 distinct question strings, which reduce to exactly 3,000 prompt groups after we ignore case, punctuation and non-ASCII characters. 2,974 of these groups (99.1%) were answered by all six models, and they account for 99.3% of the rows. In other words, the six classes were given the same prompts. This has two consequences. First, a random row-level split would put the same question in both the training and the test set, and a classifier could then be tested on topics it has already seen answered by every model. Second, the prompt should not be able to tell the models apart, which matters for RQ2.

The audit also found one exception to "the same prompts". Questions that contain an apostrophe are stored differently for different models. In Qwen's rows 64 prompts contain a real curly apostrophe, while the same questions in the other five models' rows contain a garbled three-character sequence left by an encoding error (between 55 and 65 prompts per model). As a result 65 prompt strings occur only with the label Qwen. This is a property of how the file was assembled, not of the models, and it is a small leak from the prompt to the label. It also means that grouping by the exact string would have treated these as different questions, which is why our grouping key drops non-ASCII characters. We return to this in RQ2.

Trivial giveaways. We searched every response for the names of the six model families and their developers and for phrases such as "as an AI". Table 2 shows that these are almost absent: at most 0.07% of a model's

responses mention its own family and at most 0.50% mention its own developer (Gemini responses that mention Google, a word the other models use at similar rates when the question is about technology). The models were asked subject questions and did not talk about themselves, so self-identification is not a shortcut here.

Formatting and typography. We expected markdown to be part of each model's fingerprint, but the responses are plain single paragraphs. At most 0.9% of any model's responses contain a newline and at most 2.0% contain markdown bold (Table 2), which is consistent with the whitespace cleaning the dataset authors describe. What does differ strongly is typography. Curly apostrophes appear in 47.1% of ChatGPT responses, 52.2% of DeepSeek responses and 39.7% of Qwen responses, but in almost none from Claude, Llama or Gemini, and em dashes follow a similar pattern. We cannot tell from the data whether this reflects the models themselves or the way each model's answers were copied into the file, so we treat it as a possible collection artefact and test its effect with a dedicated ablation. A related practical point is that the CSV is encoded in Windows-1252, not UTF-8.

Table 2: Audit of possible shortcuts: share of each model's responses (%) that contain the given pattern.

Share of responses (%) containing	ChatGPT	Gemini	Claude	Llama	DeepSeek	Qwen
Names its own model family	0.0	0.1	0.0	0.0	0.0	0.0
Names its own developer	0.0	0.5	0.0	0.0	0.0	0.0
Names any of the six families	0.1	0.1	0.0	0.0	0.1	0.1
AI self-reference phrase	0.0	0.0	0.0	0.0	0.0	0.0
Curly apostrophe or single quote	47.1	1.6	0.1	0.0	52.2	39.7
Straight apostrophe	11.2	56.1	54.1	42.3	20.5	11.3
Curly double quote	9.0	0.0	0.0	0.0	1.3	4.6
Straight double quote	4.7	30.9	11.6	8.0	22.9	7.0
Em dash	33.6	8.4	2.8	0.0	66.2	23.2
Newline	0.9	0.0	0.1	0.0	0.2	0.4
Markdown bold	0.0	0.0	0.5	0.0	2.0	0.0
Markdown header	0.0	0.0	0.0	0.0	0.0	0.0
Backtick	0.0	0.0	0.0	0.0	0.0	1.0
LaTeX dollar sign	0.2	6.1	0.4	0.3	1.0	0.2
No terminal punctuation at the end	1.0	1.7	0.2	2.0	3.7	0.7

2.5 Cleaning

We cleaned as little as possible. We removed the one spreadsheet-error row and the 11 duplicate responses, 12 rows in total, leaving 17,988 rows. We did not lowercase the text or remove punctuation, newlines or symbols, because these may be part of what distinguishes the models. The classes remain balanced to within eight rows.

2.6 Splits and leakage

We split the data 70/15/15 into training, validation and test sets with seed 42. Because the same prompts repeat across models, we split by prompt group instead of by row: all answers to one question go to the same split. We used scikit-learn's `StratifiedGroupKFold` with 20 folds (14 folds for training, 3 for validation, 3 for testing), which keeps the class proportions equal while keeping groups together. The result is 12,590 training, 2,699 validation and 2,699 test rows from 2,100, 450 and 450 prompt groups, and 0 groups shared between splits. The test set therefore measures whether the classifier recognises a model on questions it has never seen, not whether it remembers how each model answered a particular question. The split assignment is saved to a file and reused by every experiment. All tuning used the validation set, and each final model was evaluated on the test set once.

3 Models and Training

3.1 Tokenization and vocabulary

We use a word-level tokenizer defined by one regular expression. It emits, in order of priority, a newline, a run of letters, a run of digits, or any other single non-space character. Case is kept and every punctuation mark or symbol is its own token, so `don't` becomes three tokens and a curly apostrophe is a different token from

a straight one. We chose this over a lowercased word tokenizer because the audit showed that punctuation and typography differ between models, and over a subword tokenizer because it is simple, has no pretrained component, and makes the ablations easy to define. The vocabulary is built from the training split only: of 45,213 distinct token types we keep those that occur at least 2 times, up to 30,000, plus padding, unknown and a few special tokens, giving 30,005 entries. Tokens outside the vocabulary map to the unknown token.

Responses are truncated to 512 tokens. This covers 99.2% of training responses completely, compared with 86.4% at 384 tokens and 30.0% at 256, so truncation affects only the rare very long answers.

3.2 Convolutional network

The CNN follows Kim [2]. Token ids are mapped to 200-dimensional embeddings that are initialised randomly and learned with the rest of the network. Three one-dimensional convolutions with kernel sizes 3, 4 and 5 and 256 filters each are applied in parallel to the embedded sequence. Each is followed by a ReLU and max-pooling over time, which gives one value per filter. The three pooled vectors are concatenated into a 768-dimensional vector, passed through dropout with rate 0.3, and mapped to six logits by a linear layer. The model has 6,620,782 parameters, most of them in the embedding table. The network reads the text as overlapping 3-, 4- and 5-token windows and keeps the strongest match for each filter anywhere in the response.

3.3 Bidirectional LSTM

The recurrent model is a bidirectional LSTM [3]. It uses the same kind of 200-dimensional learned embedding, followed by dropout and a bidirectional LSTM with 2 stacked layers and hidden size 256 per direction. Sequences are packed so that padding positions are not processed. The LSTM outputs (512 dimensions per token) are summarised by max-pooling and mean-pooling over the real tokens, and the two pooled vectors are concatenated into 1024 dimensions, passed through dropout (0.5), and mapped to six logits. The model has 8,522,094 parameters. We pool over all time steps instead of using only the final hidden state because the responses are long (about 300 tokens) and evidence for a model can appear anywhere in the text.

3.4 Training procedure

Both networks are trained with the same procedure: cross-entropy loss, the AdamW optimiser with learning rate 0.001 and weight decay 0.01, and batches of 64. Gradients of the LSTM are clipped to a norm of 1. Within each epoch the training examples are shuffled and then grouped into batches of similar length, which reduces padding and halved the LSTM's epoch time; batch order is random. After each epoch we compute macro-F1 on the validation set, keep the weights of the best epoch, and stop when it has not improved for 3 epochs or after 15 epochs. Regularisation comes from dropout, weight decay and early stopping. Python, NumPy and PyTorch random seeds are fixed and cuDNN is set to deterministic mode.

Compute budget. All experiments ran on one laptop GPU (NVIDIA GeForce RTX 4060 Laptop GPU). Following our plan, we first timed one epoch of each model in its default configuration at 512 tokens (4.8 seconds for the CNN, 18.4 for the BiLSTM). Even if every required network run used all 15 epochs, the total would be about 63 minutes, well within our 2-hour budget, so we kept the maximum length at 512. That estimate left out the TF-IDF baselines and the optional BiLSTM ablations, and tuning then selected a two-layer BiLSTM that is slower per epoch. In the end the 57 training runs behind this report took 115 minutes in total: 79 for the networks and 36 for the baselines.

Hyperparameter tuning. We tuned a small grid for each model on the validation set with seed 42: the number of filters and the dropout rate for the CNN, and the hidden size and number of layers for the BiLSTM (Table 3). The configuration with the highest validation macro-F1 was used for all later experiments. The test set was not used during tuning.

Baseline. As a reference we train a logistic regression on TF-IDF features of word 1–2-grams and character 2–5-grams (case-sensitive, sublinear term frequency, 400,000 features). Its regularisation strength was chosen on the validation set ($C = 100$).

Table 3: Validation results of the tuning grid (seed 42, output-only). The chosen configurations are marked.

Model	Configuration	Parameters	Val acc.	Val F1	Best epoch	s/epoch	Chosen
CNN	128 filters, dropout 0.3	6,310,894	86.6	86.5	11	2.5	
CNN	128 filters, dropout 0.5	6,310,894	84.8	84.7	11	2.4	
CNN	256 filters, dropout 0.3	6,620,782	87.3	87.3	6	3.5	✓
CNN	256 filters, dropout 0.5	6,620,782	86.6	86.6	6	3.3	
BiLSTM	hidden 128, 1 layer	6,341,998	80.9	80.6	12	13.4	
BiLSTM	hidden 128, 2 layers	6,737,262	79.1	78.9	5	22.9	
BiLSTM	hidden 256, 1 layer	6,945,134	82.2	82.2	5	15.9	
BiLSTM	hidden 256, 2 layers	8,522,094	84.3	84.0	7	25.8	✓

4 Results

4.1 RQ1: identifying the model from its response

Table 4 reports test results for the output-only classifiers, averaged over 3 random seeds. The CNN reaches 87.5 ± 0.8% accuracy and the BiLSTM 84.6 ± 1.4%, against a chance level of 16.7%. So the answer to RQ1 is yes: on questions never seen in training, both networks name the correct model out of six for most responses. The CNN is about three points more accurate than the BiLSTM, which is roughly twice the BiLSTM's seed-to-seed standard deviation. With only 3 seeds we treat this as a likely but not certain advantage. The CNN is also much cheaper: the seed-42 run took 30 seconds, against 266 seconds for the BiLSTM.

Table 4: RQ1, output-only classification on the test set (2,699 responses). CNN and BiLSTM rows are the mean ± standard deviation over 3 seeds; the last six columns are per-class F1 (%).

Model	Accuracy	Macro-F1	ChatGPT	Gemini	Claude	Llama	DeepSeek	Qwen
CNN	87.5 ± 0.8	87.5 ± 0.8	80.5	88.0	96.6	95.4	82.4	82.0
BiLSTM	84.6 ± 1.4	84.4 ± 1.6	76.8	82.2	96.1	92.3	79.7	79.4
TF-IDF + LR	92.5	92.4	88.2	92.8	98.0	98.9	89.2	87.4
Chance	16.7	–	–	–	–	–	–	–

The TF-IDF logistic regression is clearly better than both networks, at 92.5% accuracy. We think there are two reasons. First, the networks are trained from scratch on only 12,590 examples and overfit: Figure 2 shows the training loss still falling while the validation loss stops improving after the first few epochs, and at the selected epoch the CNN has 93.9% training accuracy but 87.3% validation accuracy. Second, as Section 5 shows, much of the signal is in the frequency of particular punctuation marks, characters and words, which is exactly what a regularised linear model on character and word n-gram counts measures directly.

Easiest and hardest models. The per-class F1 scores in Table 4 split the models into two groups. Claude and Llama are the easiest to identify for both networks (for the CNN the two highest F1 scores are Claude at 96.6% and Llama at 95.4%). ChatGPT and Qwen are the hardest (ChatGPT at 80.5% and Qwen at 82.0% for the CNN; ChatGPT at 76.8% and Qwen at 79.4% for the BiLSTM). DeepSeek is almost as hard as these two, and Gemini is in between. The baseline shows the same ordering at a higher level.

Confusions. Figure 3 shows the row-normalised confusion matrices. Each row is a true model and each cell is the share of that model's test responses assigned to the column's model, so the diagonal is per-class recall. The largest off-diagonal cells for the CNN are Qwen predicted as ChatGPT (9.0%) and ChatGPT as Qwen (8.2%); for the BiLSTM they are ChatGPT as Qwen (11.8%) and DeepSeek as Gemini (10.6%). Almost all confusion is among ChatGPT, Qwen, DeepSeek and Gemini, and ChatGPT and Qwen are confused with each other in both directions.

The stylometric profiles in Section 5.2 suggest why. Claude and Llama each have an unusual profile: Claude writes by far the longest sentences (40.4 words per sentence on average, against 22.9 for Qwen), and Llama writes the shortest answers with the lowest lexical diversity. ChatGPT and Qwen, in contrast, have similar sentence lengths, similar answer lengths and the same curly-quote typography, so there is less to separate them.

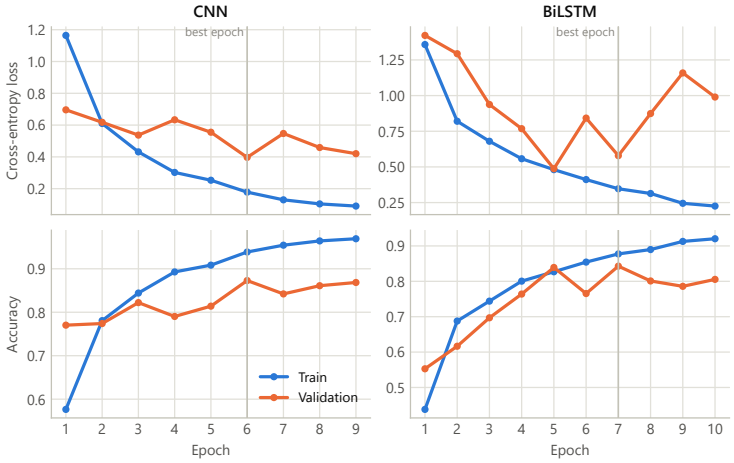


Figure 2: Training curves of the seed-42 RQ1 models. The vertical line marks the epoch with the best validation macro-F1, whose weights are used for testing.

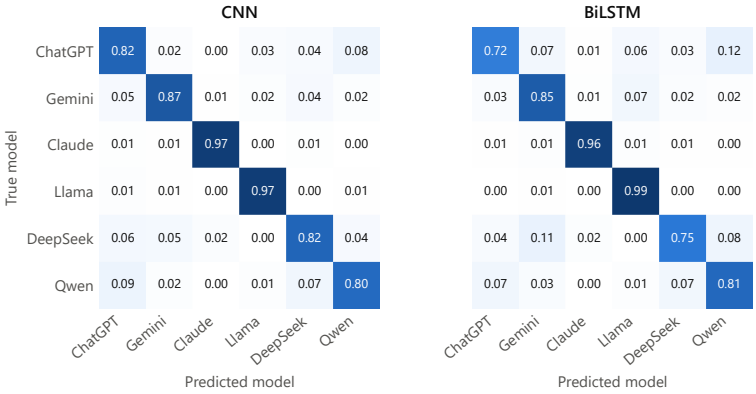


Figure 3: Row-normalised confusion matrices on the test set for RQ1, averaged over 3 seeds. Rows are true models and columns are predictions; the diagonal is per-class recall.

4.2 RQ2: does the prompt help?

We trained each model on the prompt alone, on the response alone, and on both. For the combined input we concatenate `[INP] prompt [OUT] response`, with the prompt truncated to 128 tokens (the longest prompt has 22 tokens, so nothing is cut in practice) and the response to 512 tokens as before. Table 5 gives the results.

Table 5: RQ2, test accuracy and macro-F1 (%) by input condition (seed 42). "ASCII" removes non-ASCII characters and apostrophes from the prompt before training.

	Input only		Input only (ASCII)		Output only		Input + output	
Model	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
CNN	16.8	12.6	16.5	10.9	86.6	86.6	87.5	87.5
BiLSTM	16.7	9.9	16.7	10.4	83.3	83.0	82.3	82.3
TF-IDF + LR	16.9	14.0	16.7	13.9	92.5	92.4	92.6	92.6
Chance	16.7	–	16.7	–	16.7	–	16.7	–

Can the model be predicted from the prompt alone? Essentially no. Input-only accuracy is 16.8% for the CNN, 16.7% for the BiLSTM and 16.9% for the baseline, close to the 16.7% chance level. This is what the prompt-overlap audit predicts: every model was asked the same questions, so an identical prompt appears once with each of the six labels, and whatever a classifier predicts for it can be right for only one of them.

When could input-only work, and why? The audit found one place where the prompts are not identical: questions with an apostrophe are stored with a different character encoding in Qwen's rows. This is exactly the kind of artefact that would let an input-only classifier succeed, so we measured it. We split the test rows into those whose exact prompt string is shared by all six models (2,642 rows) and the rest (57 rows, 10 of them from Qwen). On the shared prompts accuracy is 16.6% for the CNN, 16.7% for the BiLSTM and 16.7% for the baseline, which is chance. On the non-shared prompts the baseline labels 10 of the 10 Qwen rows correctly and the CNN 9, and their accuracy on these rows is 29.8% and 28.1%; the BiLSTM did not learn the cue (17.5%). So two of the three classifiers did learn that a prompt with a real curly apostrophe is a Qwen row. Because only 57 test rows are affected, this moves the overall accuracy by a fraction of a point. When we delete non-ASCII characters and apostrophes from the prompts and retrain, the cue is gone (16.5%, 16.7% and 16.7%).

This small case illustrates the general answer. If input-only classification works well, the classifier is reading how the dataset was put together (different prompt sources, templates, topics, languages or encodings for each model), not anything about the models. TXD-22 is close to a controlled experiment in this respect, so the effect is tiny. In a dataset where each model's conversations came from different users, it could be large, and it would inflate input+output results without being noticed.

Does adding the prompt to the response help? No. Compared with output-only, input+output changes accuracy by +0.9 points for the CNN, −1.1 for the BiLSTM and +0.1 for the baseline. The signs differ, and for the networks the changes are of the same size as the seed-to-seed standard deviation in Table 4 (0.8 and 1.4 points), so we read them as noise. The prompt adds about 7.8 tokens that are the same for every class.

5 In-Depth Analyses

5.1 RQ3: do fingerprints transfer across question types?

Defining domains. Our plan for RQ3 was to train on some task types (for example coding, mathematics and factual questions) and test on another (writing). TXD-22 does not allow this. It has no task column, and all its prompts are short questions about academic subjects, with no coding exercises, calculations or creative writing. We therefore define the domain by the form of the question, using rules on the first word of the prompt (the full word lists are in `src/prepare.py`). A prompt is *explanatory* if it starts with a word such as *explain, how, describe, discuss, analyze, why* or *compare*. It is *definitional* if it starts with a word such as *what, define, who, which, name* or *state* and contains none of the explanatory words. Prompts that start like a definitional question but also contain an explanatory word ("What is NAT and how does it work?") are labelled *mixed* and the few remaining ones *other*; these two groups are left out of RQ3. Table 6 gives the counts. Because every model answered the same questions, the counts are almost identical across models. We read a random sample of ten prompts per type; every prompt had the label the rules intend, but the sample also showed that "definitional" is broader than its name: it includes questions such as "What adaptations help birds fly?" that ask for more than a definition. The two types differ in the form of the question, which is a weaker kind of domain shift than a change of task.

Table 6: Question types used as domains in RQ3: rows per model (left) and distinct prompt groups per split (right).

Question type	ChatGPT	Gemini	Claude	Llama	DeepSeek	Qwen	Train	Val	Test
Definitional	2093	2087	2093	2092	2092	2093	1433	330	330
Explanatory	786	785	786	785	784	786	580	108	98
Mixed	111	111	111	112	111	111	79	11	21
Other	10	9	10	10	10	10	8	1	1

Design. We train on one question type and test on the other, in both directions. There are about 2.7 times as many definitional prompts as explanatory ones, so we subsample the definitional training prompts to the number of explanatory training prompts. Both training sets then have about 3,478 rows, and a model trained on the other type is not disadvantaged by having less data. Early stopping uses validation rows of the training type only, so the target type is never seen before testing. Each trained model is evaluated on the test rows of its own type (in-domain) and of the other type (cross-domain). For each test set, Table 7 compares the model trained on the same type with the model trained on the other type.

Table 7: RQ3. For each test set, accuracy and macro-F1 (%) of a model trained on the same question type ("In") and of a size-matched model trained on the other type ("Cross"). Drop is In minus Cross, in points.

Model	Test question type	n	In acc.	Cross acc.	Drop	In F1	Cross F1	Drop
CNN	Definitional	1981	81.8	80.4	+1.5	81.7	80.3	+1.4
CNN	Explanatory	586	83.6	83.3	+0.3	83.6	83.3	+0.3
BiLSTM	Definitional	1981	76.4	75.9	+0.6	76.3	75.7	+0.5
BiLSTM	Explanatory	586	80.7	80.2	+0.5	80.7	80.5	+0.2
TF-IDF + LR	Definitional	1981	89.0	88.3	+0.7	88.9	88.2	+0.7
TF-IDF + LR	Explanatory	586	92.2	91.0	+1.2	92.1	90.9	+1.2

Results. Performance barely decreases on the unseen question type. For the two networks the accuracy drop is between 0.3 and 1.5 points, and for all three models it is at most 1.5 points. These drops are the same size as the seed-to-seed variation we measured in RQ1, so we cannot distinguish them from zero. Table 8 gives the change in per-class F1. The changes for individual models go in both directions (from a drop of 5.6 points to a gain of 3.9 points) with no consistent pattern, which is what noise would look like. The ranking of the models is stable: Claude and Llama have a cross-domain F1 of at least 88.8% in all four network settings, and no other model exceeds 80.2%. So the same two models are the most consistently identifiable in both domains.

We conclude that, within this dataset, the classifiers learn a fingerprint that does not depend on the form of the question. This fits the other analyses below, which show that the signal lies in general style (word choice, sentence length, punctuation) and not in content. The conclusion is limited by how close our two domains are: both are single-paragraph answers to academic questions, written under the same instructions. A change to code or to creative writing could well produce a much larger drop, and this dataset cannot test that. The table also shows that the amount of data matters more than the question type: training the CNN on all 8,590 definitional rows instead of the subsample raises its accuracy on explanatory test questions from 83.3% to 88.9%.

Table 8: RQ3, change in per-class F1 (points) when the model is trained on the other question type instead of the same type. Positive values are drops.

Model	Test question type	ChatGPT	Gemini	Claude	Llama	DeepSeek	Qwen
CNN	Definitional	+2.3	+2.9	+0.6	−0.2	+2.0	+0.6
CNN	Explanatory	+0.1	+2.8	−0.6	−0.5	−0.9	+1.1
BiLSTM	Definitional	+5.6	+3.1	−2.0	−2.6	−0.3	−0.6
BiLSTM	Explanatory	−1.0	+0.4	+4.5	+3.9	−3.9	−2.7

5.2 RQ4: stylometric profiles

To see what distinguishes the models, we computed 38 stylometric features for every response, in five groups: *length* (characters, words, sentences, words per sentence); *lexical* (mean word length, type-token ratio, moving-average type-token ratio (MATTR) with a 50-word window [5], pronoun rates, share of uppercase letters and of digits); *punctuation* (rates per 100 words of commas, periods, semicolons, colons, question and exclamation marks, parentheses, hyphens and slashes); *typography* (curly and straight apostrophes and quotes, en and em dashes, share of non-ASCII characters); and *formatting* (markdown headers, bullets, numbered lists, bold, code and tables, LaTeX markers, emoji, newline density, and whether the response ends with terminal punctuation). MATTR is included because the plain type-token ratio falls as a text gets longer, so it partly measures length.

Figure 4 shows the mean of the most informative features for each model. Each model has a recognisable profile. Claude writes the longest answers (298 words on average) in very long sentences (40.4 words per sentence) with long words. Llama writes the shortest answers (210 words) and has clearly the lowest lexical diversity (MATTR 0.765, against 0.878 for DeepSeek). DeepSeek uses the most parentheses (2.12 per 100 words, against 0.24 for Llama) and em dashes (0.64 per 100 words). Gemini uses the most semicolons and colons, and Qwen the most commas. ChatGPT, DeepSeek and Qwen use curly apostrophes and the other three models do not. The formatting features are almost always zero because the responses contain almost no markdown.

A classifier on the features alone. We trained two classifiers on only these 38 numbers, with the same split. Logistic regression reaches 72.3% test accuracy and gradient boosting (scikit-learn's histogram-based implementation with default settings) reaches 79.7%. So a few dozen hand-made style statistics, with no access

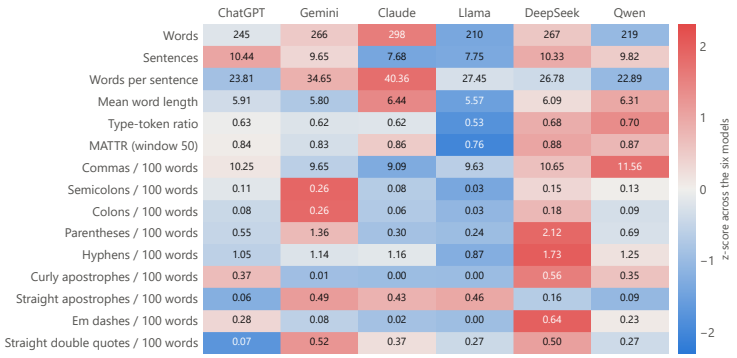


Figure 4: Mean value of selected stylometric features for each model. Numbers are the means; colour is the z-score of the mean across the six models (blue below average, red above).

to the words themselves, recover most of what the CNN achieves (87.5%). By permutation importance on the validation set, the five most important features for the gradient-boosting model are the type-token ratio (shuffling it lowers accuracy by 14.1 points), curly apostrophes / 100 words (7.8), characters (6.8), parentheses / 100 words (5.9) and MATTR (window 50) (5.6).

Table 9 shows which kind of feature carries the signal. Each of the length, lexical and punctuation groups alone gives between 55.2% and 59.0% accuracy, and typography alone gives 42.4%. Removing any one of these four groups costs only 4.0 to 5.4 points, because the others compensate. The formatting group alone is near chance (20.1%) and removing it changes nothing. The signal is therefore spread over several kinds of features, both structural (length, punctuation, typography) and lexical, and no single kind is necessary.

Table 9: Gradient boosting on stylometric features: test accuracy (%) when using one feature group only, and when using all features except that group.

Feature group	Features	Only this group	All except it	Loss when removed
Length	4	59.0	75.1	+4.6
Lexical	7	55.2	74.3	+5.4
Punctuation	9	55.5	75.7	+4.0
Typography (quote and dash style)	7	42.4	75.7	+4.0
Formatting (markdown, newlines)	11	20.1	79.4	+0.3
All features	38	79.7	–	–

Distinctive n-grams and phrases. Table 10 lists, for each model, the unigrams and bigrams that are most over-represented in its training responses compared with the other five models, ranked by the log-odds ratio with an informative Dirichlet prior [4]. Because all models answered the same questions, topic words cancel out and what remains is style. ChatGPT is marked by curly double quotes and by the word “especially”. Gemini is marked by the dollar sign (it writes formulas in LaTeX), parentheses, colons and semicolons. Claude is marked by connecting words such as “that”, “through”, “while” and “including” and by clauses introduced with “, creating”, which matches its long sentences. Llama is marked by “can”, “used to” and “, such” (as in “such as”), a plainer explanatory style. DeepSeek is marked by parentheses, em dashes and “e.g.”, and Qwen by commas, “for example” and “fostering”. We also looked at how responses begin and end. Openings restate the topic of the question and are not distinctive: no three-word opening accounts for more than 1.7% of any model’s responses. Closings are more telling (last column of Table 10): 8.7% of ChatGPT responses end with a sentence starting “In summary”, and 6.1% of Llama responses with one starting “By understanding”.

5.3 RQ4: ablations

The feature analysis shows what differs between models. To test what the neural classifiers actually need, we modified the responses in the training, validation and test sets in the same way, retrained the output-only models from scratch, and measured the change in test accuracy (Table 11). All ablations use seed 42 and are compared with the seed-42 RQ1 model, so differences below roughly two points are within seed noise. The modifications are: (a) remove markdown syntax; (b) replace every run of whitespace, including newlines, with one space; (c)

Table 10: Most distinctive n-grams per model (training split, lowercased; each box is one n-gram). Punctuation marks are tokens, so `. in` means a sentence that starts with "In". The last column gives the most common first two words of the final sentence, with the share of that model's responses.

Model	Most over-represented n-grams (highest z-score first)	Last sentence starts with
ChatGPT	`. in` `“` (curly) `”` (curly) `. it` `’` (curly) `’` (curly) `s` `especially` `it`	in summary (8.7%)
Gemini	`$` `a` `(` `:` `)` `the` `;` `'` (straight)	despite these (2.7%)
Claude	`that` `through` `while` `including` `creating` `'` (straight) `'` (straight) `s` `, creating`	despite these (1.3%)
Llama	`can` `a` `. by` `the` `, such` `of` `used` `used to`	by understanding (6.1%)
DeepSeek	`)` `(` — (em dash) `. ,` `. g` `g .` `e .` `g`	ultimately the (2.1%)
Qwen	`,` `. for` `\` `*` `example ,` `for example` `fostering` `example`	despite these (2.9%)

lowercase the text and delete all punctuation and symbols; (d) keep only the first 64 or 128 tokens; (e) replace the names of the six model families and their developers with a placeholder; and (f), which we added after the audit, map curly quotes, long dashes and similar characters to plain ASCII.

Table 11: RQ4 ablations. Test accuracy (%) after retraining on modified responses and the change from the unmodified model in points (Δ). "Responses changed" is the share of responses that the modification alters.

	Responses	CNN		BiLSTM		TF-IDF + LR	
Responses modified	changed (%)	Acc.	Δ	Acc.	Δ	Acc.	Δ
Original text	–	86.6	–	83.3	–	92.5	–
(a) Markdown stripped	2.5	87.1	+0.5	82.6	−0.7	92.1	−0.4
(b) Whitespace normalised	0.6	86.8	+0.2	83.1	−0.2	92.6	+0.1
(c) Lowercase, no punctuation	100.0	80.9	−5.7	77.3	−6.0	90.4	−2.1
(d) First 128 tokens	99.1	75.8	−10.8	73.4	−9.9	81.2	−11.3
(d) First 64 tokens	100.0	64.4	−22.3	60.8	−22.5	68.3	−24.2
(e) Model names masked	0.8	85.5	−1.1	83.4	+0.1	92.4	−0.1
(f) Typography normalised	35.7	87.3	+0.7	82.3	−1.0	91.9	−0.6
(c)+(d) Words only, first 64	100.0	58.0	−28.6	56.4	−26.9	64.7	−27.8

Formatting, whitespace and self-identification do not matter here. Ablations (a), (b) and (e) change accuracy by at most 1.1 points in either direction, which is noise. This is expected from the audit, because they alter only 2.5%, 0.6% and 0.8% of the responses. The dataset simply does not contain these signals.

Typography is not needed. Ablation (f) changes 35.7% of the responses, yet accuracy stays within about one point of the unmodified models (CNN 87.3%, BiLSTM 82.3%, baseline 91.9%). In the audit, curly apostrophes looked like a giveaway, and typography features alone do give 42.4% accuracy, but the classifiers do not depend on them: when they are removed, other cues carry the same information. This matters for the validity of the results, because typography was the cue most likely to come from the collection process instead of the models.

Case and punctuation help moderately. Ablation (c) removes every structural cue except length and leaves only a sequence of lowercase words. Accuracy falls by 5.7 points for the CNN (to 80.9%), 6.0 points for the BiLSTM (to 77.3%) and 2.1 points for the baseline (to 90.4%). Punctuation and case therefore contribute, but most of the performance survives on words alone.

Truncation hurts most. With only the first 128 tokens the CNN reaches 75.8%, and with the first 64 tokens 64.4%; the BiLSTM and the baseline behave the same way. Almost every response is longer than 64 tokens (only 0.01% are shorter), so at 64 tokens the length of the response is hidden from the classifier. However, truncation also removes about four fifths of the text, so this drop mixes two effects: the loss of the length cue and the loss of evidence. The feature analysis gives a rough idea of the first effect: length features alone give 59.0% accuracy, but removing them from the full feature set costs only 4.6 points. We therefore think most of the drop is lost

evidence: the fingerprint is weak in any single sentence and accumulates over the response. Even the hardest condition, the first 64 lowercase words with no punctuation, gives 58.0% for the CNN, more than three times chance.

Structural, lexical, or both? Both, with the lexical part larger. Word choice and phrasing alone give the CNN 80.9% of its 86.6% (seed 42), punctuation and case add about 5.7 points, length is informative but largely redundant with the other cues, and formatting contributes nothing in this dataset because it was removed before release.

5.4 Error analysis

We compared the test errors of the seed-42 CNN and BiLSTM. The CNN misclassifies 361 of the 2,699 test responses and the BiLSTM 450. They share 216 errors, and in 144 of those they predict the same wrong model. The two architectures therefore fail on many of the same responses, but each also makes many errors the other does not, which suggests that combining them could help. Misclassified responses are shorter than correctly classified ones (275 against 304 tokens on average for the CNN, 277 against 305 for the BiLSTM), consistent with the truncation result that less text means less evidence.

Table 12 shows one example for each of the four most frequent confusions that both models made. The examples were selected by rule (the first such test row), not by hand. In the first one a Qwen response opens with a parenthetical remark, which is DeepSeek's most distinctive habit in Table 10, and both networks label it DeepSeek. The other three open with a neutral definitional sentence that contains none of the markers from Table 10 and that any of the confused models could have written. This seems typical of the errors among ChatGPT, Qwen and DeepSeek: a response is misclassified when it happens to use another model's habit, or when it uses none of its own.

Table 12: Examples of test responses misclassified in the same way by both the CNN and the BiLSTM (seed 42).

True	Predicted	Prompt (italic) and start of the response
Qwen	DeepSeek	*How does a trie data structure work?* A trie (pronounced "try"), also known as a prefix tree, is a tree-like data structure used to store a dynamic...
DeepSeek	Gemini	*What is perfect competition? List its features.* Perfect competition is a theoretical market structure characterized by a complete absence of monopolistic...
ChatGPT	Qwen	*What is a socket? How does socket programming work?* A socket is an endpoint for communication between two devices over a network, facilitating data exchange...
Qwen	ChatGPT	*What is virtual memory?* Virtual memory is a memory management technique that allows an operating system to use disk space as an...

6 Lessons and Experience

What worked. Auditing the data before modelling was the most useful step. It showed that all models answered the same questions, that the responses had no markdown, and that the file had encoding problems, and each of these changed the design: the split by prompt group, the choice of ablations, and the way we read the file. Training a simple baseline alongside the networks was also valuable, because it gave a reference that the networks did not beat. On the engineering side, saving every run's metrics to a file and generating all tables and numbers in this report from those files meant that rerunning the pipeline could not leave stale numbers in the text, and grouping training batches by length halved the BiLSTM's training time.

What did not work, and surprises. The first surprise was that both networks lost to TF-IDF with logistic regression (87.5% and 84.6% against 92.5%). With about twelve thousand training examples and randomly initialised embeddings, the networks overfit, and the task rewards exactly the frequency statistics a linear model captures. The second was that the BiLSTM was both slower (about seven times the CNN's training time) and less accurate than the CNN; a recurrent model's ability to follow long-range order did not pay off for a task that depends on local patterns. The third was typography: it looked like the strongest cue in the audit but removing it changed nothing, which taught us that a large difference in a marginal statistic does not show that a classifier relies on it; only an ablation does. Tuning was less informative than we hoped, since the differences between grid points in Table 3 are of the same size as the differences between seeds.

We also made a mistake that cost a full rerun. Our first grouping key treated a question stored with a

curly apostrophe and the same question stored with a garbled apostrophe as two different prompts, so a few dozen questions ended up in different splits for Qwen and for the other models. We noticed it only because an input-only model scored slightly above chance, which should be impossible when every model gets the same prompts. After fixing the key and rebuilding the split, the input-only models are at chance, as Table 5 shows. The lesson is to treat any result that is better than it should be as a sign of leakage and to check group identity on normalised text, not raw strings.

Limitations. The main limitation is the dataset. All results come from one collection with one prompt style (short academic questions), one answer format (a single paragraph of at most about 300 words) and formatting that was flattened before release. The dataset description does not state the model versions, the generation settings, or whether responses came from an API or a chat interface, and the responses date from early 2025. A classifier trained here identifies "the way this model's answers look in TXD-22", which may mix properties of the model with properties of how its answers were collected; the typography differences are an example, although the ablation shows our results do not depend on them. We do not know how well the classifiers would work on other prompts, other versions of the same models, or raw chat output with markdown. RQ3 tests only a mild shift (question form), because the data has no real task variety. Apart from RQ1, each experiment was run with one seed, so differences under about two points should not be interpreted. Our truncation ablation does not cleanly separate the length cue from the amount of evidence. Finally, the BiLSTM was sometimes still improving when it reached the 15-epoch limit, so it may be slightly undertrained.

With more time. We would (i) use pretrained representations, either pretrained word embeddings for the CNN and BiLSTM or a fine-tuned transformer, which we expect to close the gap to the baseline; (ii) run every experiment with several seeds and report confidence intervals; (iii) test on a different dataset with real task variety; (iv) collect our own responses with known model versions and the original markdown, to measure how much of the fingerprint is formatting; and (v) separate length from evidence by sampling fixed-length windows at random positions instead of always taking the start.

7 Conclusion

On six model families from TXD-22, a classifier can identify which LLM wrote a response far better than chance: 87.5% for a CNN, 84.6% for a BiLSTM and 92.5% for a TF-IDF logistic regression, tested on questions that were never seen in training. Claude and Llama are easy to recognise, while ChatGPT, Qwen and DeepSeek are often confused with each other. The prompt carries no information because all models were asked the same questions, and the one prompt cue we found came from an encoding artefact. The fingerprint transfers between definitional and explanatory questions with no measurable loss. It is mostly lexical (word choice and phrasing) with a smaller contribution from punctuation and case; it is spread over the whole response and gets stronger with length; and it does not depend on markdown, self-identification or typographic characters. These conclusions hold for this dataset; data with real task variety and known model versions is the next step.

Use of AI tools. As the project description permits, an LLM assistant (Claude) was used to write and debug the code and to draft this report, under the author's direction. All experiments were run on the author's machine, every number in the report is generated from the saved result files, and the author reviewed the code, results and text and is responsible for them.

References

[1] M. S. Iqbal and M. A. Kashem. *TXD-22: A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification*. Mendeley Data, version 1, 2026. `https://doi.org/10.17632/prcjcggtjf.1`

[2] Y. Kim. Convolutional neural networks for sentence classification. In *Proceedings of EMNLP*, pages 1746–1751, 2014.

[3] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[4] B. L. Monroe, M. P. Colaresi, and K. M. Quinn. Fightin' words: Lexical feature selection and evaluation for identifying the content of political conflict. *Political Analysis*, 16(4):372–403, 2008.

[5] M. A. Covington and J. D. McFall. Cutting the Gordian knot: The moving-average type-token ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100, 2010.